

面向对象程序设计基础

期末复习资料

刘知远 2026 春

By TOM

涵盖 14 讲课件 • OJ 6 次作业 • 3 套往年期末题

目录

第一章 C++ 语言基础	5
1.1 编译与链接	5
1.1.1 声明与定义	5
1.1.2 main 函数参数	5
1.2 宏定义	6
1.2.1 宏的典型用途	6
1.3 内联函数	6
1.3.1 内联 vs 宏的对比	6
1.4 Makefile	6
1.5 GDB 调试	7
第二章 函数重载与默认参数	8
2.1 函数重载	8
2.1.1 自动类型转换与重载	8
2.2 默认参数	8
2.2.1 默认参数与重载的二义性	8
2.3 auto 与 decltype	9
2.3.1 auto	9
2.3.2 decltype	9
2.3.3 追踪返回类型	9
第三章 类与封装	10
3.1 类的定义	10
3.2 访问权限	10
3.3 this 指针	10
3.4 构造函数	11
3.4.1 初始化列表	11
3.4.2 默认构造函数	11
3.5 析构函数	11
3.5.1 构造与析构顺序	12
3.5.2 全局与局部对象的生命周期	12
3.6 对象的 new 和 delete	12
第四章 运算符重载与引用	13
4.1 引用	13

4.1.1	引用 vs 指针	13
4.1.2	引用作参数与返回值	13
4.2	运算符重载	13
4.2.1	成员函数 vs 全局函数	13
4.2.2	++ 前缀与后缀的区别	13
4.2.3	流运算符重载	14
4.2.4	下标运算符 [] 重载	14
第五章	静态成员、常量与友元	15
5.1	静态成员	15
5.2	常量成员	15
5.3	友元	15
5.4	类型转换	16
第六章	拷贝与移动语义	17
6.1	拷贝构造函数	17
6.1.1	拷贝构造的三种调用时机	17
6.1.2	浅拷贝问题	17
6.2	右值引用与移动语义	17
6.2.1	左值与右值	17
6.2.2	移动构造函数	18
6.2.3	std::move	18
6.2.4	拷贝赋值与移动赋值	18
6.2.5	Rule of Five	18
6.2.6	返回值优化 (RVO)	18
第七章	组合与继承	20
7.1	组合 (has-a 关系)	20
7.2	继承 (is-a 关系)	20
7.2.1	构造顺序与基类构造调用	20
7.2.2	访问权限表	20
7.3	重写隐藏	20
7.3.1	using 关键字的多种用途	21
第八章	虚函数与多态	22
8.1	向上类型转换与对象切片	22
8.2	虚函数与晚绑定	22
8.2.1	虚函数表 (VTABLE)	22
8.2.2	虚析构函数	22

8.2.3	构造函数/析构函数中的虚函数	23
8.2.4	const 对虚函数重写的影响	23
8.3	纯虚函数与抽象类	23
8.4	多态的条件	23
第九章	模板与泛型	25
9.1	函数模板	25
9.1.1	模板的编译期实例化	25
9.2	类模板	25
9.3	静态多态 vs 动态多态	25
第十章	命名空间与 STL	27
10.1	命名空间	27
10.2	STL 容器	27
10.2.1	序列容器	27
10.2.2	关联容器	27
10.2.3	容器选择原则	27
10.3	迭代器	27
10.3.1	迭代器失效	27
10.4	string	27
第十一章	IO 流、函数对象与智能指针	29
11.1	IO 流	29
11.2	函数对象	29
11.3	智能指针	29
11.3.1	shared_ptr (引用计数)	29
11.3.2	unique_ptr (独占所有权)	29
第十二章	设计模式	31
12.1	行为型模式	31
12.1.1	模板方法 (Template Method)	31
12.1.2	策略模式 (Strategy)	31
12.2	结构型模式	31
12.3	创建型模式	31
附录 A	OJ 题目考点速查	33

模块 1 C++ 语言基础

1.1 编译与链接

将源代码变为可执行程序需两步：**编译**将 .cpp 转为 .o 目标文件，**链接**将目标文件与库文件合并为可执行文件。

```
1 g++ -c file.cpp -o file.o # 只编译不链接
2 g++ file.o -o file      # 链接
3 g++ a.cpp b.cpp -o program # 一步完成
```

1.1.1 声明与定义

声明告诉编译器名称和类型（不分配内存）；**定义**则实际分配内存。函数可以有多处声明，但只能有一处定义。这一区别对于理解多文件编程至关重要。

```
1 extern int x;      // 变量声明（不分配内存）
2 int x;             // 变量定义（分配内存）
3 int add(int a, int b); // 函数声明
4 int add(int a, int b) { return a + b; } // 函数定义
```

注意：int x; 即便没有显式初始化，在全局作用域下也是定义（会被默认初始化为 0）。

1.1.2 main 函数参数

main(int argc, char** argv) 是程序入口。其中 argc 表示命令行参数的个数（包括程序名本身），argv 是一个字符串数组，argv[0] 是程序名，argv[1] 及之后是真正的命令行参数。

```
1 // 假设运行：./program 4 5
2 // argc = 3
3 // argv = {"/program", "4", "5"}
4 int main(int argc, char** argv) {
5     if (argc != 3) { // 先检查参数个数
6         cerr << "Usage: " << argv[0] << " op1 op2" << endl;
7         return 1;
8     }
9     int a = atoi(argv[1]); // 字符串转整数
10    int b = atoi(argv[2]);
11    cout << a + b << endl;
12    return 0;
13 }
```

注意：访问 argv 之前必须先检查 argc，否则可能发生越界访问。

1.2 宏定义

宏由预处理器在编译前处理，做纯文本替换。宏不具备类型检查功能，且可能产生边界效应。

```
1 #define PI 3.14159
2 #define SQR(x) ((x)*(x)) // 正确写法：外层括号+参数括号
3 #define MIN(a,b) ((a)<(b)?(a):(b))
```

注意：SQR(1+2) 若写成 #define SQR(x) x*x，展开后变为 1+2*1+2=5，与期望的 (1+2)*(1+2)=9 不符。因此宏参数必须加括号，整个表达式也必须加括号。

1.2.1 宏的典型用途

- 防止头文件重复包含：#ifndef ... #define ... #endif 或 #pragma once
- 条件编译：#ifdef DEBUG 控制调试信息的输出
- 定义常量和简单函数（但 C++ 中用 const 和 inline 替代更安全）

1.3 内联函数

内联函数在调用点直接展开代码，避免函数调用的压栈、跳转、返回等开销。

```
1 inline int max(int a, int b) { return a > b ? a : b; }
```

1.3.1 内联 vs 宏的对比

- 类型检查**：内联函数做类型检查，宏不做
- 可调试性**：内联函数在 Debug 版本不真正内联，可设断点调试；宏不可调试
- 边界效应**：内联函数不会出现宏的多次求值问题（如 MAX(a++, b)）
- 访问权限**：内联函数可以访问类的私有成员，宏不能
- 建议性**：inline 关键字对编译器只是建议，编译器可能忽略

注意：定义在类内的成员函数默认为内联函数。应避免对大段代码或包含循环的函数使用 inline。多文件编译时内联函数的实现必须写在头文件中。

1.4 Makefile

Makefile 定义了目标、依赖关系和生成命令，只有依赖文件比目标文件更新时才会重新执行命令。

```
1 target: prerequisites
2 \tcommand          # 前导必须是Tab
3 # 自动变量：
4 # $@ 目标文件名 $^ 所有依赖文件 $< 第一个依赖文件
```

```
1 # 示例
2 all: test
3 test: main.o func.o
4      g++ main.o func.o -o test
```

```
5 main.o: main.cpp
6     g++ -c main.cpp -o main.o
7 func.o: func.cpp
8     g++ -c func.cpp -o func.o
9 clean:
10     rm *.o test
```

1.5 GDB 调试

```
1 g++ -g file.cpp -o file # 编译时加入调试信息
2 gdb file                # 启动GDB调试
3 b 10                     # break 在第10行设断点
4 r                       # run 运行至断点
5 n                       # next 单步（不进入函数）
6 s                       # step 单步（进入函数）
7 p x                     # print 输出变量x的值
8 c                       # continue 继续运行至下一断点
9 watch x                 # 监视变量x的变化
10 quit                   # 退出gdb
```

模块 1 知识关系

核心主线：从源程序到可执行文件的过程（编译 → 链接），引出多文件协作的机制（声明与定义的区别），进而引出如何组织多文件项目（Makefile）和如何在出现问题后进行调试（GDB）。

宏与内联的对比：两者都旨在“代码复用”，但宏是预处理期的纯文本替换（危险但灵活），内联是编译期的类型安全优化。这个对比是理解“编译时 vs 运行时”思维模式的起点。

OJ 对应：HW1 选择题和 Makefile 练习、HW1 帮小明 debug（GDB 调试）。

模块 2 函数重载与默认参数

2.1 函数重载

函数重载允许在同一作用域中存在多个同名函数，只要它们的**参数列表**不同（包括参数类型或参数个数）。**返回值类型不能作为重载的区分依据**。

```
1 void print(const char* msg);
2 void print(int score);
3 void print(int score, int times); // 重载OK: 参数个数不同
4 int print(int score);           // 错误! 仅返回值不同, 无法区分
```

编译器根据调用时的实参类型进行重载解析。当存在多个候选函数时，优先选择参数类型精确匹配的函数；若没有精确匹配，再尝试自动类型转换（如 `int` 转 `float`、`char` 转 `int` 等）。

2.1.1 自动类型转换与重载

```
1 void print(int score) { cout << "int"; }
2 void print(float score) { cout << "float"; }
3 int main() {
4     float a = 1.0;
5     print(a); // float = 1.0 → 精确匹配 print(float), 输出 "float"
6     print(1); // int → 精确匹配 print(int), 输出 "int"
7 }
```

当有精确匹配时，不会进行类型转换。

2.2 默认参数

函数参数可以在声明时指定默认值。**有默认值的参数必须放在参数列表的末尾**。

```
1 void show(char* name, int score=0, char* msg="pass");
2 show("Tom");           // OK, score=0, msg="pass"
3 show("Tom", 90);       // OK, msg="pass"
4 show("Tom", 90, "good"); // OK
```

2.2.1 默认参数与重载的二义性

默认参数与函数重载结合时可能产生二义性问题，导致编译错误。

```
1 void fun(int a, int b=1) { }
2 void fun(int a) { }
3 fun(2); // 编译错误! 两个函数都匹配: fun(int,int)可用默认参数, fun(int)也匹配
```


2.3 auto 与 decltype

2.3.1 auto

编译器根据初始化表达式自动推导变量类型。**auto 变量必须在定义时初始化，且必须在编译期确定类型。**

```
1 auto i = 3;           // int
2 auto f = 4.0f;        // float
3 auto a('c');          // char
4 auto* x = new auto(3); // int*
5 auto b4 = 10, b5 = 20.0; // 错误！推导为不同类型
```

注意：auto 不是一个真正的类型，不能用于 sizeof 或 typeid。

2.3.2 decltype

decltype 根据表达式推导类型，可用于重用匿名类型。

```
1 struct { int d; double b; } anon_s;
2 decltype(anon_s) as; // 定义与 anon_s 同类型的变量
```

2.3.3 追踪返回类型

```
1 auto func(int x, int y) -> decltype(x+y) {
2     return x + y;
3 }
4 // C++14中可简写为：
5 auto func(int x, int y) { return x + y; }
```

模块 2 知识关系

函数重载是 C++ 区别于 C 的核心特性之一，为“同一操作适应不同类型”提供了语法支持。默认参数进一步增强了函数的灵活性。两者结合时需要注意**二义性**问题——这是考试选择题的常见考点。

auto 和 decltype 是 C++11 引入的类型推导机制，为后续的模板编程（自动推导返回类型）和范围 for 循环（for (auto x : vec)）奠定基础。

模块 3 类与封装

3.1 类的定义

类是用户定义的类型，包含**成员变量**（数据）和**成员函数**（操作）。类的声明通常放在头文件（.h）中，成员函数的实现放在源文件（.cpp）中。

```
1 // agent.h — 头文件中声明
2 class Agent {
3     std::string memory[6]; // 默认private
4 public:
5     void plan(std::string goal);
6 };
7 // agent.cpp — 实现文件中定义
8 void Agent::plan(std::string goal) {
9     // 实现...（类外定义需用类名::限定）
10 }
```

3.2 访问权限

- **private**: 仅类内可以访问（类的缺省访问权限）
- **public**: 任何地方都可以访问
- **protected**: 类内和派生类中可以访问

这一机制实现了 OOP 的核心原则之一：**隐藏实现，提供服务**。类的设计者将接口暴露为 public，将实现细节隐藏在 private 中，使用者只需要关心“能做什么”而不需要知道“怎么做的”。

```
1 class Agent {
2     int id; // private（默认）
3 public:
4     void setID(int i) { id=i; } // public接口
5 private:
6     void internal(); // 内部实现，对外隐藏
7 };
8 Agent a;
9 a.setID(10); // OK, 通过public接口访问
10 a.id = 10; // 错误! private成员不可访问
```

3.3 this 指针

每个成员函数中都隐含一个指向当前对象的指针参数，名为 **this**。

```
1 void Agent::plan(std::string goal) {
2     this->memory[0] = goal; // 等价于 memory[0] = goal
3 }
```

这是成员函数与普通函数的本质区别——成员函数多了一个隐含的 `this` 参数。

注意：静态成员函数没有 `this` 指针，因此静态成员函数不能访问非静态成员变量，也不能调用非静态成员函数。这是考试中判断静态函数能否访问某成员的依据。

3.4 构造函数

构造函数在对象创建时自动调用，用于初始化对象。**构造函数无返回值、函数名与类名相同、可以重载。**

```
1 class Student {
2     int ID;
3 public:
4     Student() : ID(0) {}           // 默认构造函数
5     Student(int id) : ID(id) {}    // 带参构造函数
6     Student(int year, int order) : ID(year*10000+order) {}
7 };
```

3.4.1 初始化列表

初始化列表位于构造函数参数列表之后、函数体之前，以冒号开头。

```
1 Student(int id) : ID(id) { } // 初始化列表
```

初始化顺序严格按照成员在类中声明的顺序，而不是在初始化列表中的顺序！

```
1 class Student {
2     int ID1;           // 先声明
3     int ID2;           // 后声明
4 public:
5     Student(int id) : ID2(id), ID1(ID2) { }
6     // 实际执行顺序：先初始化ID1（此时ID2未初始化，值随机）
7     //                再初始化ID2（=id）
8 };
```

3.4.2 默认构造函数

不带参数的构造函数，或所有参数都有默认值的构造函数。

```
1 Student() = default; // C++11: 显式要求编译器生成默认构造函数
2 Student(char) = delete; // 显式禁用某个构造函数
```

如果用户定义任何构造函数，编译器不再隐式生成默认构造函数。

3.5 析构函数

析构函数在对象销毁时自动调用，用于释放资源。**名称：类名、无参数、一个类只能有一个析构函数。**

```
1 class Classroom {  
2     int* ID_list;  
3 public:  
4     ~Classroom() { if (ID_list) delete[] ID_list; }  
5 };
```

3.5.1 构造与析构顺序

- **构造顺序**：成员变量（按声明顺序）→ 自身构造函数体
- **析构顺序**：自身析构体 → 成员变量（反向声明顺序）

3.5.2 全局与局部对象的生命周期

- **局部对象**：在定义时构造，离开作用域时析构
- **全局对象**：在 main() 函数之前构造，在 main() 返回之后析构。同一编译单元中按照定义顺序初始化。不同编译单元之间的初始化顺序不确定
- **静态局部对象**：首次执行到该对象的定义代码时构造，离开作用域时**不析构**，程序结束时才析构

3.6 对象的 new 和 delete

```
1 A* p = new A(42); // 分配内存 + 调用构造函数  
2 delete p;        // 调用析构函数 + 释放内存  
3 A* arr = new A[10]; // 分配数组（多4字节存储数组大小）  
4 delete[] arr;     // 必须用 delete[]!
```

new[] 必须配 delete[]，否则只会调用一次析构函数（只析构第一个元素），并且释放的地址不对（new[] 分配的地址偏移了 4 字节，delete[] 会自动调整地址）。

模块 3 知识关系

本模块是 OOP 的基石。**封装**（private/public）实现了”隐藏实现、提供服务”——这是 OOP 区别于面向过程编程的关键。**构造函数和析构函数**保证了对象自动初始化和自动清理，避免了忘记初始化或忘记释放资源的常见错误。

初始化列表不仅语法上更高效（避免先默认构造再赋值），而且是初始化 const 成员和引用成员的**唯一途径**。**构造/析构顺序**是 OOP 考试中代码输出题的高频考点。

知识连线：封装 → 构造/析构自动管理 → new/delete 动态创建 → 生命周期管理（局部/全局/静态）。这四个知识点层层递进，构成了 C++ 对象管理的基础框架。

OJ 对应：各次作业中均涉及构造/析构的输出预测题。

模块 4 运算符重载与引用

4.1 引用

引用是变量的别名。引用必须在定义时初始化，且不能修改引用指向，不存在空引用。

```
1 int i = 1;
2 int& j = i; // j 是 i 的引用
3 j = 2;      // 等价于 i = 2
```

4.1.1 引用 vs 指针

- 引用必须在定义时初始化，指针可以延迟初始化
- 引用一旦绑定不能改指向，指针可以随时指向不同对象
- 不存在空引用，但可以有空指针 (nullptr)
- 引用用起来更安全（编译器接管了解引用操作）
- 引用不是对象，不能建立引用的引用

4.1.2 引用作参数与返回值

引用作为函数参数可以避免拷贝开销，同时允许函数修改实参。

引用作为返回值可以支持链式调用（如 `cout << a << b`），但不能返回局部变量的引用。

4.2 运算符重载

运算符重载使自定义类型可以像内置类型一样使用运算符。

4.2.1 成员函数 vs 全局函数

```
1 A operator+(A a, A b) { return A(a.data + b.data); } // 全局函数
2 class A {
3     A operator+(A b) { return A(this->data + b.data); } // 成员函数
4 };
```

只能通过成员函数重载的运算符：`=` `[]` `()` `->`

原因：以 `=` 为例，如果允许全局重载 `=`，编译器将无法确定是否应该自动合成默认的拷贝赋值运算符，导致标准行为被破坏。而 `[]`，`()`，`->` 与类对象的“身份”紧密相关，应当由类自己控制。

++/- 的前后缀区分：后缀重载函数有一个 `int` 哑元参数用于区分，该哑元在函数体中不需要使用（可以省略变量名）。

4.2.2 ++ 前缀与后缀的区别

```
1 class Test {
2 public:
3     int data;
4     Test& operator++() { ++data; return *this; } // 前缀: 返回引用
5     Test operator++(int) { Test t(data); ++data; return t; } // 后缀: 返回副本
6 };
7 // 前缀语义: 先++后赋值; 后缀语义: 先赋值后++
8 int a = ++b; // b先+1, 再赋给a
9 int a = b++; // 先赋给a, b再+1
```

4.2.3 流运算符重载

必须使用**全局函数**重载，且返回引用以支持链式调用。

```
1 ostream& operator<<(ostream& out, const Test& t) {
2     out << t.data;
3     return out;
4 }
5 istream& operator>>(istream& in, Test& t) {
6     in >> t.data;
7     return in;
8 }
```

4.2.4 下标运算符 [] 重载

```
1 int& operator[] (const char* name); // 可用字符串作下标
2 // 返回引用使得 obj["key"] = value; 得以成立
```

模块 4 知识关系

引用是 C++ 区别于 C 的关键特性——它提供了比指针更安全的间接访问方式，是后续拷贝构造、拷贝赋值、移动语义等机制的基础。

运算符重载是”自定义类型用起来像内置类型”的核心手段。**++** 前后缀的区别（哑元参数区分）和流运算符必须全局函数重载的限制是考试常考点。

OJ 对应：HW2 大整数类考查了多种运算符重载。其中 **++/+=/«** 是必须掌握的基本操作。

模块 5 静态成员、常量与友元

5.1 静态成员

静态成员**属于类本身而不属于某个具体对象**，因此所有对象共享同一份静态数据。

```
1 // Test.h
2 class Test {
3     static int count;    // 声明：在头文件中
4 public:
5     Test() { count++; }
6     ~Test() { count--; }
7 };
8 // Test.cpp
9 int Test::count = 0;    // 定义：必须在实现文件中进行！
```

静态成员的访问：既可以通过对象（`a.count`）也可以通过类名（`Test::count`）。

静态成员函数不能访问非静态成员，因为静态函数没有 `this` 指针，无法确定要访问哪个对象的成员。

5.2 常量成员

```
1 class Student {
2     const int ID;          // 常量数据成员
3     const int age = 9;     // 就地初始化（C++11）
4 public:
5     Student(int id) : ID(id) {} // 必须在初始化列表中初始化
6     int getID() const { return ID; } // 常量成员函数
7 };
```

常量成员函数（函数声明末尾加 `const`）承诺不修改对象的状态。

常量对象只能调用常量成员函数。

常量静态变量：`int` 和 `enum` 类型可以就地初始化，其他类型需要在类外定义。

注意：不存在“常量静态函数”（`static const`），因为静态函数属于类（不需要绑定到对象），而 `const` 需要绑定到对象（决定对象能否调用）。

5.3 友元

被声明为友元的函数或类可以访问该类的所有成员（包括 `private` 和 `protected`）。

```
1 class A {
2     int data;
3     friend void foo(A& a); // 友元函数
4     friend class B;        // 友元类
5 };
```

```
6 void foo(A& a) { cout << a.data; } // 可以访问私有成员!
```

友元的三条重要性质：

- **非对称性**：A 声明 B 为友元，不等于 B 声明 A 为友元
- **不传递性**：朋友的朋友不是你的朋友
- **不继承性**：朋友的孩子不是你的朋友

注意：友元声明可以放在类的 `private` 或 `public` 区域，效果相同——友元不受访问权限区域的影响。另外，`friend class Y {};` 是错误语法（不能在友元声明中定义类）。

5.4 类型转换

类型转换有两种方法，**两种方法不能同时使用**。

- 方法一：在源类中定义 `operator DstType() const`
- 方法二：在目标类中定义转换构造函数 `DstType(const SrcType& s)`

模块 5 知识关系

“静态成员控制”类维度”的属性和行为（如记录对象个数）。**const 成员**控制对象的”不可变性”。友元控制”访问权限的例外”——它在封装性上开了一个可控的口子。**类型转换**实现不同类之间的自动桥接。

四个知识点共同服务于同一个主题：对类成员（包括函数和数据）的**精细化访问控制**。

OJ 对应：各次作业中 `const` 成员函数的调用权限判断是常见考点。

模块 6 拷贝与移动语义

本章是 OOP 考试的核心考点，在每次作业和期末考试中都会涉及。

6.1 拷贝构造函数

拷贝构造函数是一种特殊的构造函数，参数是本类对象的常量引用 `const ClassName&`。

```
1 class Agent {
2     char* memory; int m_size;
3 public:
4     Agent(const Agent& src) : m_size(src.m_size) {
5         memory = new char[m_size];    // 深拷贝: 重新分配内存
6         for(int i=0; i<m_size; i++) memory[i] = src.memory[i];
7     }
8 };
```

6.1.1 拷贝构造的三种调用时机

1. 用一个类对象初始化另一个新的类对象: `Agent b(a);` 或 `Agent b = a;`
2. 函数调用时以类的对象为形参（值传递）: `func(Agent a)`
3. 函数返回类对象（值返回）: `Agent func() { return Agent(); }`

6.1.2 浅拷贝问题

如果使用编译器隐式生成的拷贝构造函数，对于指针成员会进行**位拷贝（浅拷贝）**，导致两个对象的指针指向同一块内存（拷贝构造机制见第17页）。析构时两个对象先后 `delete` 同一块内存，造成**双删（double free）**，程序崩溃。

```
1 Agent a(5);
2 Agent b = a; // 默认位拷贝→a.memory 和 b.memory 指向同一地址
3 // 析构时: 先析构b→delete memory; 再析构a→delete 同一 memory (崩溃!)
```

凡是有指针成员类，必须自定义拷贝构造函数实现深拷贝（第??页），或者用 `= delete` 禁用拷贝。

6.2 右值引用与移动语义

6.2.1 左值与右值

- **左值**：可以取地址、有名字的值（如变量、引用）
- **右值**：不能取地址、没有名字的值（如字面量、表达式临时结果、函数返回值）

```
1 int a = 1;
2 int& r = a;    // OK, 左值引用绑定左值
3 int& r2 = a + 1; // 错误! a+1 是右值
```

```

4 const int& r3 = a + 1; // OK, const左值引用可以绑定右值
5 int&& rr = a + 1; // OK, 右值引用绑定右值 (C++11)

```

6.2.2 移动构造函数

移动构造函数”偷取”临时对象的资源，避免深拷贝（第??页）的开销。

```

1 Agent(Agent&& t) : memory(t.memory), m_size(t.m_size) {
2     t.memory = nullptr; // 将原对象指针置空
3     t.m_size = 0;      // 原对象不再持有资源
4 }

```

移动后的对象必须处于”可析构”的状态（即 `delete nullptr` 是安全的）。

6.2.3 std::move

`std::move` 不执行任何移动操作，它只是将左值转换为右值引用，使得移动构造/移动赋值可以被触发。

```

1 Agent a;
2 Agent b = std::move(a); // 触发移动构造（而不是拷贝构造）

```

6.2.4 拷贝赋值与移动赋值

```

1 Agent& operator=(const Agent& right); // 拷贝赋值
2 Agent& operator=(Agent&& right); // 移动赋值

```

赋值运算符必须检查**自赋值**: `if (this == &right) return *this;`

6.2.5 Rule of Five

如果类需要显式定义析构函数（通常是因为有指针成员），那么它通常也需要显式定义（浅拷贝问题见第??页）：

1. 拷贝构造函数
2. 拷贝赋值运算符
3. 移动构造函数（C++11）
4. 移动赋值运算符（C++11）
5. 析构函数

以上这些函数即使不定义，编译器也会按需自动合成隐式版本。

6.2.6 返回值优化（RVO）

编译器在默认情况下会进行**返回值优化**（Return Value Optimization），避免函数返回局部对象时的拷贝/移动构造开销。在 RVO 作用下，局部对象会直接构造在返回值接收者的内存地址上。

```

1 Agent func() {
2     Agent loc; // 局部对象
3     return loc; // RVO: loc 直接构造在接收者的地址上

```

```

4 }
5 int main() {
6     Agent a = func(); // 不会调用拷贝/移动构造!
7 }
    
```

RVO 的条件: return 的值类型与函数返回值类型相同, 且 return 的是一个局部对象。使用编译选项 `-fno-elide-constructors` 可以禁用 RVO 以观察完整的拷贝/移动调用链。

模块 6 知识关系

问题	解决方案
默认拷贝构造函数对指针成员做浅拷贝, 导致双删	自定义拷贝构造函数实现深拷贝
深拷贝在频繁拷贝时开销太大	引入移动构造函数, 转移资源所有权
左值只能触发拷贝, 无法触发移动	<code>std::move</code> 将左值转为右值引用
移动后原对象状态不确定	移动构造函数中将原对象指针置空

知识链条: 浅拷贝问题 → 引入深拷贝 → 深拷贝频繁开销大 → 引入移动语义 → `std::move` 配合使用 → Rule of Five 最佳实践。

OJ 对应: HW2 回文字符串与深拷贝 (深拷贝实现)、HW2 大整数类 (拷贝 + 移动)、2024 期末计算图 (`shared_ptr` 代替手动内存管理)。

模块 7 组合与继承

7.1 组合（has-a 关系）

当一个类的对象是另一个类的组成部分时，称之为组合关系（has-a）。

```
1 class AgentCompany {
2     CEOAgent ceo; // AgentCompany "有一个" CEOAgent
3     CTOAgent cto;
4 };
```

子对象在构造函数的初始化列表中初始化，按声明顺序构造，先于当前对象（构造顺序详见第??页）。析构顺序相反。

7.2 继承（is-a 关系）

```
1 class CEOAgent : public Agent { ... };
```

不继承：构造函数（C++11 中用 `using` 可以继承）、析构函数、赋值运算符、友元函数。

7.2.1 构造顺序与基类构造调用

```
1 CEOAgent(string s) : Agent(s) { } // 显式调用基类构造函数
2 CEOAgent() { } // 隐式调用基类默认构造函数
```

7.2.2 访问权限表

基类中的访问权限	public 继承	protected 继承	private 继承
public	public	protected	private
protected	protected	protected	private
private	不可访问	不可访问	不可访问

public 继承体现 **is-a** 关系（基类能用之处派生类也能用）。private 继承体现 **is-implementing-in-terms-of**（实现复用），不能向上转换。

7.3 重写隐藏

派生类中定义与基类同名的函数时，如果函数参数不同或者基类函数不是虚函数，就会**隐藏**基类的所有同名函数（包括所有重载版本）。

```
1 class Agent {
2 public:
3     void setTemp() { }
4     void setTemp(int i) { }
```

```
5 };
6 class CEOAgent : public Agent {
7 public:
8     void setTemp(int i) { } // 隐藏了基类的 setTemp() 和 setTemp(double)
9 };
10 CEOAgent d;
11 d.setTemp(0.9); // 调用 CEOAgent::setTemp(int), 类型转换 0.9→1
12 // d.setTemp(); // 错误! 基类的无参版本被隐藏
```

使用 `using Agent::setTemp;` 可以恢复被隐藏的基类函数。

7.3.1 using 关键字的多种用途

- 引入命名空间成员: `using std::cout;`
- 引入整个命名空间: `using namespace std;`
- 继承基类构造函数 (C++11): `using Base::Base;`
- 恢复被隐藏的基类函数: `using Base::func;`
- 定义类型别名 (C++11): `using IntPtr = int*;`

模块 7 知识关系

组合 vs 继承 是 OOP 设计的核心选择: 组合 (has-a) 表达”整体-部分”, 继承 (is-a) 表达”一般-特殊”。

知识链条: 组合的构造顺序 → 继承的构造顺序 (基类 → 派生类) → 访问权限表 → 重写隐藏 → **using** 恢复。拷贝构造规则 (第17页) 同样适用于组合中的子对象。

OJ 对应: HW3 装配交通工具 (组合 + 继承)、2025 期末简单的软件工程团队 (继承 + 多态)。

模块 8 虚函数与多态

8.1 向上类型转换与对象切片

派生类指针/引用转换为基类指针/引用称为**向上类型转换**（编译器自动完成，仅 public 继承有效）。

当派生类的对象**按值传递**给接受基类对象的函数时，会发生**对象切片**——派生类中新增的数据成员被“切掉”。

```
1 void func(Agent a) { a.speak(); } // 值传递→切片，调用 Agent::speak
2 void func(Agent& a) { a.speak(); } // 引用传递→无切片，多态有效
```

注意：避免对象切片的方法：用基类指针或引用而不是值来传递参数。

8.2 虚函数与晚绑定

virtual 关键字开启**晚绑定**（运行时多态）。通过基类指针或引用调用虚函数时，实际执行的是对象真实类型对应的函数版本。

注意：若基类函数已声明为 **virtual**，则派生类中同名同参的函数无论是否加 **virtual** 都是虚函数。但如果派生类函数与基类虚函数**参数不同**，则构成重写隐藏而非覆盖，虚函数表不会被覆盖。

```
1 class Agent {
2 public:
3     virtual void plan() { cout << "Agent plan"; }
4 };
5 class CEOAgent : public Agent {
6     void plan() override { cout << "CEO plan"; }
7 };
8 void exec(Agent& a) { a.plan(); }
9 CEOAgent ceo;
10 exec(ceo); // 输出 "CEO plan" — 多态！
```

8.2.1 虚函数表（VTABLE）

每个包含虚函数的类都有一个**虚函数表（VTABLE）**，其中存储了该类所有虚函数的入口地址。每个对象中包含一个**虚函数指针（VPTR）**，指向所属类的 VTABLE。

sizeof(类) 中，带虚函数的类比不带虚函数的多一个指针大小（8 字节在 64 位系统上）。多个虚函数共享同一个 VPTR。

8.2.2 虚析构函数

基类的析构函数应该声明为 virtual。否则，通过基类指针删除派生类对象时，只会调用基类的析构函数（析构顺序见第??页），导致派生类部分资源泄漏。

```

1 class Agent { public: virtual ~Agent() {} }; // 虚析构
2 class CEOAgent : public Agent { ... };
3 Agent* p = new CEOAgent();
4 delete p; // 先 ~CEOAgent(), 再 ~Agent() — 正确释放

```

8.2.3 构造函数/析构函数中的虚函数

在构造函数和析构函数中调用虚函数时，**虚机制不工作**——调用的是当前类的“本地版本”。原因是基类构造时派生类部分尚未初始化，如果此时调用派生类的虚函数可能会访问未初始化的数据。

8.2.4 const 对虚函数重写的影响

const 是函数签名的一部分，影响虚函数的重写覆盖。基类虚函数若带 const，派生类的覆盖版本也必须带 const，否则构成重写隐藏而非覆盖。

```

1 class Base {
2     virtual void f() const { cout << "Base"; }
3 };
4 class Der : public Base {
5     void f() { cout << "Der"; } // 隐藏而非覆盖（缺少 const）
6 };
7 // 结果: Der::f 不会覆盖 Base::f, 通过基类指针调用时仍执行 Base::f

```

这是考试中容易被忽略的陷阱。

8.3 纯虚函数与抽象类

```

1 virtual void work() = 0; // 纯虚函数

```

包含至少一个纯虚函数的类称为**抽象类**，不能实例化。抽象类的主要用途是定义接口规范，强制派生类实现特定的函数。

8.4 多态的条件

继承 + 虚函数 + 指针/引用

- 通过对象直接调用函数 → 编译期绑定（早捆绑）→ 不产生多态
- 通过指针/引用调用虚函数 → 运行时绑定（晚捆绑）→ 产生多态

模块 8 知识关系

概念	说明
对象切片	值传递派生类给基类参数时，派生类特有数据丢失。 解决：用指针/引用
早捆绑	编译期确定函数地址。非虚函数 + 对象直接调用
晚捆绑	运行期确定函数地址。虚函数 + 指针/引用调用
VTABLE	每类一张虚函数表。编译器在构造函数中设置 VPTR
虚析构	基类析构必须是 virtual，否则删除派生类对象时只调基类析构
纯虚函数	<code>=0</code> 声明，使类成为抽象类，用于定义接口

知识链条： 向上类型转换（基础）→ 对象切片（问题）→ 早捆绑的局限性 → virtual 开启晚捆绑 → VTABLE/VPTR 实现机制 → 虚析构防止内存泄漏 → 纯虚函数定义抽象接口。

OJ 对应： HW3/HW4 多态应用，2024 期末计算图（虚函数 + 智能指针），2025 期末软件团队（多态实现协同工作）。

模块 9 模板与泛型

9.1 函数模板

```
1 template<typename T> T sum(T a, T b) { return a + b; }
2 cout << sum(1, 2); // T 推导为 int
3 cout << sum(1.0, 2.0); // T 推导为 double
4 // sum(1, 2.0);      // 错误! 无法推导 (T 冲突)
5 cout << sum<int>(1, 2.0); // 手动指定 T=int, 2.0 被转换为 int
```

9.1.1 模板的编译期实例化

模板在**编译期**实例化，编译器根据实际使用的类型生成对应的代码。这意味着**模板的声明和定义必须在同一个文件中**（通常放在头文件中），否则链接时找不到实例化代码。

注意：template<typename T> 中的 **typename** 和 **class** 可以互换。但在模板内部表示“类型”时，**必须使用 typename**（例如在依赖类型前：typename T::iterator 中的 **typename** 不可省略，否则编译器会将其解析为静态成员变量）。

9.2 类模板

```
1 template<typename T, unsigned N> class Array {
2     T data[N];
3 public:
4     T& operator[](int i) { return data[i]; }
5 };
6 Array<int, 10> arr;
```

非类型参数（int、enum、指针等）必须在编译期确定，不能用变量。

```
1 int n = 5;
2 // Array<char, n> arr0; // 错误! n 是变量
3 const int m = 5;
4 Array<char, m> arr1; // OK, m 是常量
5 Array<char, 5> arr2; // OK
```

9.3 静态多态 vs 动态多态

- **静态多态（模板）**：编译期确定类型，无虚函数调用开销，但编译后代码增多
- **动态多态（虚函数）**：运行时确定类型，灵活方便，但有函数调用开销

模块 9 知识关系

模板是 C++ 中“代码复用”的最高形式——它允许同一段代码作用于不同类型。理解模板必须在头文件中实现（编译期实例化需要看到完整定义）是常考概念。

OJ 对应：2024 期末图书管理系统（类模板）、2025 期末高维数组（N 维类模板 + 迭代器实现）。

模块 10 命名空间与 STL

10.1 命名空间

命名空间用于避免命名冲突。标准库的所有内容都在 `std` 命名空间中。

```
1 namespace A { int x; }
2 A::x = 3;
3 using A::x;           // 引入部分成员
4 using namespace A;    // 引入全部（可能污染命名空间）
```

10.2 STL 容器

10.2.1 序列容器

- **vector**: 动态数组，内存连续，随机访问 $O(1)$ ，自动扩容（capacity 倍增）
- **list**: 双向链表，任意位置高速插入/删除，不支持随机访问

10.2.2 关联容器

- **set**: 有序不重复元素集合，底层红黑树，find/insert/erase $O(\log n)$
- **map**: 键值对集合，下标运算符在 key 不存在时自动创建新元素

10.2.3 容器选择原则

- 频繁随机访问 → vector
- 频繁中间插入/删除 → list
- 唯一有序元素 → set
- 键值映射 → map

10.3 迭代器

迭代器为不同容器提供统一的遍历接口，其核心是 `operator++` 和 `operator*` 的重载（见第??页）。`begin()` 返回第一个元素的迭代器，`end()` 返回最后一个元素之后的位置（左闭右开区间）。

10.3.1 迭代器失效

- **vector**: insert/erase 使操作位置之后的所有迭代器失效；引起扩容的操作使所有迭代器失效
- **list**: 只有被删除元素的迭代器失效，插入操作不导致任何迭代器失效

10.4 string

```
1 string s = "hello";
2 s.size(); s.length(); // 字符串长度
3 s.c_str();           // 转为 C 风格字符串 (const char*)
4 s.substr(pos, len);  // 取子串
5 s.find("ll");        // 查找, 返回位置或 string::npos
6 getline(cin, s);     // 读入一行
7 to_string(42);       // "42"
8 stoi("42");          // 42
```

注意：字符串拼接用 `+=` ($O(n)$)，不要用 `+` 循环拼接 ($O(n^2)$)。

模块 10 知识关系

STL 将”数据结构”（容器）和”算法”解耦，通过迭代器作为桥梁。容器选择、迭代器失效和关联容器特性是考试中 STL 部分的三大考点。

OJ 对应：HW2 键值对（map）、HW5 容器查询（set+ 函数对象）、HW6 发工资（STL 综合）、2024 期末文本编辑器（string+list）。

模块 11 IO 流、函数对象与智能指针

11.1 IO 流

```
1 ifstream ifs("input.txt"); // 文件输入流
2 ifs >> x; getline(ifs, line);
3 stringstream ss;          // 字符串流（内存中读写）
4 ss << 100 << " " << 200;
5 int a, b; ss >> a >> b; // a=100, b=200
6 ss.str();                // 获取完整 buffer 内容
```

11.2 函数对象

重载 `operator()`（运算符重载见第??页）的类可以像函数一样被调用。

```
1 struct Greater {
2     bool operator()(int a, int b) const { return a > b; }
3 };
4 sort(v.begin(), v.end(), Greater()); // 自定义函数对象
5 sort(v.begin(), v.end(), greater<int>()); // STL 预定义
```

11.3 智能指针

11.3.1 `shared_ptr`（引用计数）

`shared_ptr` 通过引用计数来管理资源，多个 `shared_ptr` 可以共享同一对象，其拷贝/赋值行为类似于指针的深拷贝（第17页）。当最后一个 `shared_ptr` 被销毁或重置时，引用计数归零，自动 `delete` 所管理的对象。使用 `make_shared<T>(...)` 创建更高效（一次内存分配）。

```
1 shared_ptr<int> p1 = make_shared<int>(42);
2 cout << p1.use_count(); // 1
3 { shared_ptr<int> p2 = p1; // 引用计数变为2
4     cout << p1.use_count(); } // 2
5 cout << p1.use_count(); // 1 (p2 出作用域，计数减回1)
6 // p1 出作用域时计数归0，自动 delete int
```

11.3.2 `unique_ptr`（独占所有权）

`unique_ptr` 表示独占所有权，不能拷贝只能移动。适合表示“唯一持有”的资源，开销与裸指针相同。

```
1 unique_ptr<Agent> p1(new Agent());
2 unique_ptr<Agent> p2 = std::move(p1); // 转移所有权
3 // p1 现在为空
```

模块 11 知识关系

`stringstream` 提供了灵活的字符串 $\leftrightarrow$ 数值转换。函数对象（特别是比较函数）是 STL 算法自定义行为的核心接口。智能指针（第??页）解决了手动管理内存的痛点——其引用计数原理本身就体现了 Proxy（代理）设计模式的思想。

OJ 对应： HW5 Argument Parser（`stringstream` 应用）、2024 期末计算图（`shared_ptr` 管理节点内存）。

模块 12 设计模式

12.1 行为型模式

12.1.1 模板方法 (Template Method)

在抽象基类中定义算法的骨架（模板方法），将一些步骤的实现延迟到子类中。体现”开放封闭原则”：对扩展开放（添加新子类），对修改封闭（不改基类代码）。

```
1 class Workflow {
2 public:
3     void run() { plan(); execute(); report(); } // 骨架
4     virtual void plan() = 0;
5     virtual void execute() = 0;
6     virtual void report() = 0;
7 };
```

12.1.2 策略模式 (Strategy)

定义一系列算法并封装，使它们可以互相替换。算法的变化独立于使用算法的客户。体现”单一责任原则”。

```
1 class Strategy { public: virtual void doIt() = 0; };
2 class StrategyA : public Strategy { void doIt() { /*A*/ } };
3 class Context { Strategy* s; public: void run() { s->doIt(); } };
```

12.2 结构型模式

- **适配器 (Adapter)**：接口转换。让不兼容的类可以协同工作
- **代理 (Proxy)**：接口不变，增加控制。如智能指针的引用计数
- **装饰器 (Decorator)**：链式包裹，动态增加功能。如先后添加安全和日志功能

12.3 创建型模式

- **单例 (Singleton)**：保证全局唯一实例 + 统一访问接口
- **工厂方法 (Factory Method)**：封装单个对象的创建逻辑
- **抽象工厂 (Abstract Factory)**：创建一组相关对象，保证产品族内兼容

模块 12 知识关系

三个维度的分类：**行为型**（关注算法/职责）、**结构型**（关注类/对象组合）、**创建型**（关注对象创建）。三者互不冲突，可以组合使用。

模式对比：

- 模板方法（继承 + 骨架）vs 策略（组合 + 可替换）
- 适配器（改接口）vs 代理（加控制）vs 装饰器（链式加功能）
- 工厂方法（单产品）vs 抽象工厂（产品族）

OJ 对应： HW6 架构设计题（策略/模板方法）、2023 期末多级字典（抽象工厂/策略 + 模板）。

A OJ 题目考点速查

来源	题目	核心考点
HW1	选择题	面向对象基本概念
HW1	Makefile	多文件编译、Makefile 语法
HW1	帮小明 debug	GDB 调试、常见 Bug
HW2	回文字符串与深拷贝	拷贝构造、深拷贝 vs 浅拷贝
HW2	简易键值对容器	STL map、pair
HW2	大整数类	运算符重载 (++,«)
HW3	计算图形面积	继承、虚函数、多态
HW3	装配交通工具	组合与继承、构造顺序
HW4	模拟动物	纯虚函数、抽象类
HW4	计算图	运算符重载、模板
HW4	代码审阅	代码阅读与改错
HW5	容器查询	STL、函数对象
HW5	Argument Parser	iostream、stringstream
HW6	发工资	STL 综合
HW6	架构设计 ×2	Template Method、Strategy
2023 期末	多级字典	模板 + 策略 + 多级结构
2024 期末	图书管理	类模板
2024 期末	文本编辑器	string、list、迭代器
2024 期末	计算图	多态 + shared_ptr
2025 期末	软件团队	继承 + 多态 + 虚函数
2025 期末	草莓巧克力	继承 + STL + 多态
2025 期末	高维数组	N 维模板 + 迭代器实现

By TOM · 祝考试顺利!